

گزارش پروژه ریاضیات گسسته

بخش رمز ویزتر

VIGENERE CIPHER

A	B	C	D	E	F	G
L	E	M	O	N	L	E

شماره دانشجویی

40424193

نویسنده

پارسا هاشمی سعادت

بخش رمز ویژتر - پروژه ۹

در این بخش رمز ویژتر پیاده‌سازی شده است. علاوه بر رمزگذاری و رمزگشایی با کلید، امکان شکستن رمز بدون کلید، تحلیل آماری متن و اجرای آزمایش نرخ موفقیت هم وجود دارد.

مثال ساده رمزگذاری و رمزگشایی

● رمزگذاری با کلید معلوم

فرض کنیم می‌خواهیم متن `ATTACKATDAWN` را با کلید `LEMON` رمز کنیم. چون کلید از متن کوتاه‌تر است، آن را تا رسیدن به طول متن تکرار می‌کنیم:

```

متن اصلی:  A T T A C K A T D A W N
کلید تکراری: L E M O N L E M O N L E
متن رمز:   L X F O P V E F R N H R
  
```

برای هر ستون، شماره حرف متن اصلی و شماره حرف کلید را با هم جمع می‌کنیم و حاصل را پیمانه ۲۶ می‌گیریم. چند ستون اول:

```

A = 0   L = 11   0 + 11 = 11 → L
T = 19  E = 4    19 + 4 = 23 → X
T = 19  M = 12   19 + 12 = 31 → 31 mod 26 = 5 → F
A = 0   O = 14   0 + 14 = 14 → O
C = 2   N = 13   2 + 13 = 15 → P
  
```

همین کار برای بقیه حروف ادامه پیدا می‌کند و نتیجه می‌شود:

```

ATTACKATDAWN → LXFOPVEFRNHR
  
```

فاصله‌ها و نشانه‌هایی مانند ویرگول و علامت سؤال رمز نمی‌شوند و در جای خود باقی می‌مانند.

● رمزگشایی با کلید معلوم

حالا فرض کنیم متن رمز `LXFOPVEFRNHR` و کلید `LEMON` را داریم. برای رمزگشایی، کلید را دوباره تا رسیدن به طول متن رمز تکرار می‌کنیم:

```

متن رمز:      L X F O P V E F R N H R
کلید تکراری:  L E M O N L E M O N L E
متن بازیابی: A T T A C K A T D A W N

```

در رمزگذاری شمارهٔ حرف کلید را با شمارهٔ حرف متن اصلی جمع کردیم. برای رمزگشایی عمل برعکس را انجام می‌دهیم: شمارهٔ حرف کلید را از شمارهٔ حرف رمز کم می‌کنیم و نتیجه را پیمانهٔ ۲۶ می‌گیریم. چند حرف اول:

```

L = 11   L = 11   11 - 11 = 0 → A
X = 23   E = 4    23 - 4  = 19 → T
F = 5    M = 12   5 - 12 = -7 → -7 mod 26 = 19 → T

```

پس متن اصلی بازیابی می‌شود:

```

ATTACKATDAWN

```

پیاده سازی

کد به چند قسمت جدا تقسیم شده است:

- `vigenere_cipher.py` برای رمزگذاری و رمزگشایی است.
- `vigenere_cryptanalysis.py` محاسبات مربوط به شکستن رمز را انجام می دهد.
- `text_statistics.py` آمار متن و نمودارها را می سازد.
- `experiment.py` برای اجرای چند آزمایش با طول های مختلف استفاده می شود.

ساختار فایل‌ها

- رمزگذاری و رمزگشایی با کلید معلوم: `src/vigenere_cipher.py`
- شاخص تطابق، آزمون کاسیسکی، تخمین طول کلید، بازیابی کلید و شکستن رمز: `src/vigenere_cryptanalysis.py`
- جدول و نمودار فراوانی حرف، دوگرام و سه‌گرام: `src/text_statistics.py`
- آزمایش نرخ موفقیت رمزشکنی برحسب طول متن: `src/experiment.py`
- رابط خط فرمان برای اجرای همه بخش‌ها: `main.py`
- تست‌های خودکار: `tests/test_vigenere.py`
- پیکره نمونه برای آزمایش: `data/sample_plaintext.txt`

مدلسازی ریاضی و اثبات‌ها

در محاسبات، حروف انگلیسی با اعضای مجموعهٔ پیمانه‌ای زیر نمایش داده می‌شوند:

```

Z26 = {0, 1, 2, ..., 25}

```

مدل می‌شوند؛ یعنی $A = 0$ ، $B = 1$ ، ... و $Z = 25$. فاصله، عدد، نشانه‌ها و حروف غیرانگلیسی وارد این مدل نمی‌شوند و در برنامه بدون تغییر باقی می‌مانند.

۱. مدل ریاضی رمزگذاری و رمزگشایی

فرض کنید متن اصلی دنبالهٔ $P_0, P_1, \dots, P_{(n-1)}$ و کلید دارای طول m و برابر دنبالهٔ $K_0, K_1, \dots, K_{(m-1)}$ باشد. کلید به‌صورت دوره‌ای تکرار می‌شود؛ بنابراین حرف کلید مورد استفاده در جایگاه i برابر $K_{(i \bmod m)}$ است.

```

Encryption: Ci = (Pi + K(i mod m)) mod 26
Decryption: P'i = (Ci - K(i mod m)) mod 26

```

اثبات درستی رمزگشایی

فرمول رمزگذاری را داخل فرمول رمزگشایی قرار می‌دهیم:

```

P'_i = ((P_i + K_(i mod m)) - K_(i mod m)) mod 26
      = P_i mod 26
      = P_i

```

چون P_i از ابتدا عددی بین ۰ و ۲۵ است، $P_i \bmod 26$ همان P_i خواهد بود. پس برای تمام جایگاه‌ها $P'_i = P_i$ و در نتیجه:

```

Decrypt(Encrypt(P, K), K) = P

```

این همان دلیل ریاضی تست رفت‌وبرگشت رمزگذاری و رمزگشایی در پروژه است.

● ۲. چرا Brute Force عملی نیست؟

اگر طول کلید m معلوم باشد، برای هر یک از m جایگاه کلید ۲۶ انتخاب مستقل وجود دارد. طبق اصل ضرب، تعداد کلیدهای ممکن برابر است با:

$$|K_m| = 26 \times 26 \times \dots \times 26 = 26^m$$

اگر طول کلید نیز معلوم نباشد و همه طول‌های ۱ تا M امتحان شوند، تعداد کلیدها برابر مجموع هندسی زیر است:

$$26^1 + 26^2 + \dots + 26^M = (26^{(M+1)} - 26) / 25$$

رشد این تعداد نمایی است. چند نمونه:

زمان خوش‌بینانه با یک میلیارد کلید در ثانیه

طول کلید تعداد کلیدهای ممکن

حدود ۰/۰۱۲ ثانیه $26^5 = 11,881,376$ ۵

حدود ۳/۵ دقیقه $26^8 = 208,827,064,576$ ۸

بیش از ۳ سال $26^{12} = 95,428,956,661,682,176$ ۱۲

حدود ۱/۳۸ میلیون سال $26^{16} = 43,608,742,899,428,874,059,776$ ۱۶

عدد 26^m فقط تعداد کلیدهای ممکن را نشان می‌دهد. برای امتحان کردن هر کلید، برنامه باید تمام n حرف متن را با آن رمزگشایی کند و سپس خروجی را بررسی کند. بنابراین مقدار کار تقریباً برابر است با:

$$\begin{aligned} &\text{تعداد کلیدها} \times \text{تعداد حروف متن} \\ &= 26^m \times n \end{aligned}$$

مثلاً اگر طول کلید $m=4$ و طول متن $n=1000$ باشد، تعداد کلیدهای ممکن $26^4=456976$ است و برای هر کدام باید حدود ۱۰۰۰ حرف بررسی شود؛ یعنی حدود ۴۵۷ میلیون پردازش حرف. در نماد پیچیدگی این مقدار را $O(n \times 26^m)$ می‌نویسند. پس روش brute force فقط برای کلیدهای بسیار کوتاه عملی است.

روش آماری پروژه همه کلیدهای کامل را امتحان نمی‌کند. ابتدا طول کلید را تخمین می‌زند، سپس هر حرف کلید را جداگانه مانند یک کلید سزار با ۲۶ احتمال بررسی می‌کند. بنابراین برای کلیدی با طول m ، به جای 26^m کلید کامل، تقریباً $26 \times m$ انتقال بررسی می‌شود.

● ۳. مدل شاخص تطابق (Index of Coincidence)

اگر در متنی با N حرف، فراوانی حرف j برابر f_j باشد، شاخص تطابق از رابطه زیر محاسبه می‌شود:



```
IC = sum(f_j * (f_j - 1)) / (N * (N - 1))
```

اثبات فرمول IC

از میان N حرف، یک حرف را انتخاب می‌کنیم. $N \times (N - 1)$ زوج مرتب از دو جایگاه متفاوت وجود دارد. برای حرف j نیز $f_j \times (f_j - 1)$ زوج مرتب وجود دارد که هر دو عضو آن حرف j هستند. با جمع‌کردن این مقدار برای تمام ۲۶ حرف و تقسیم بر تعداد کل زوج‌ها، احتمال یکسان‌بودن دو حرف تصادفی به دست می‌آید؛ این احتمال دقیقاً همان IC است.

مقدار $1/26$ مربوط به یک **مدل تصادفی یکنواخت** است: هر بار یک حرف به‌صورت مستقل انتخاب می‌شود و هر ۲۶ حرف احتمال یکسان دارند. برای فهم آن ابتدا یک الفبای چهارتایی را در نظر بگیر:



```
الفبا = {A, B, C, D}
```

اگر دو حرف مستقل انتخاب کنیم، ۱۶ جفت هم‌احتمال ممکن داریم:



```
AA AB AC AD
BA BB BC BD
CA CB CC CD
DA DB DC DD
```

فقط چهار جفت زیر دارای دو حرف یکسان‌اند:



```
AA BB CC DD
```

پس در این مثال:



```
احتمال یکسان بودن = 4 / 16 = 1 / 4
```

برای الفبای انگلیسی نیز همین منطق برقرار است. تعداد تمام جفت‌های ممکن $26 \times 26 = 676$ است و ۲۶ جفت یکسان داریم:



```
AA, BB, CC, ..., ZZ
```


بنابراین:



احتمال یکسان بودن = $26 / 1 = 676 / 26 = 0.0385$

پس منظور این نیست که در هر متن محدود، مقدار IC دقیقاً $1/26$ است. اگر از دو جایگاه متفاوت یک متن واقعی نمونه بگیریم، مقدار دقیق به تعداد واقعی هر حرف بستگی دارد و با فرمول IC محاسبه می‌شود. عدد $1/26$ مقدار مورد انتظار برای متن تصادفی یکنواخت است و IC متن‌های یکنواخت بلند معمولاً به آن نزدیک می‌شود.

در زبان انگلیسی حروف توزیع یکنواخت ندارند و IC معمولاً نزدیک 0.066 است. همین اختلاف امکان تشخیص ستون‌های شبیه متن انگلیسی را فراهم می‌کند.

چرا IC طول کلید را آشکار می‌کند؟

فرض کن کلید سه حرف دارد و بارها تکرار می‌شود:



کلید: D O G D O G D O G ...

پس حرف‌های شماره ۱، ۴، ۷ و ... متن همگی با **D** رمز شده‌اند. حرف‌های شماره ۲، ۵، ۸ و ... همگی با **O** و حرف‌های شماره ۳، ۶، ۹ و ... همگی با **G** رمز شده‌اند.

اگر برنامه طول کلید را درست و برابر ۳ حدس بزند، این سه گروه را جدا می‌کند. داخل هر گروه همه حروف با یک انتقال یکسان رمز شده‌اند؛ بنابراین الگوی تکرار حروف انگلیسی هنوز دیده می‌شود و IC آن گروه نسبتاً بالا می‌ماند.

اگر برنامه مثلاً طول را به اشتباه ۲ حدس بزند، حروفی که با **D**، **O** و **G** رمز شده‌اند با هم در یک گروه قرار می‌گیرند. الگوی فراوانی آن‌ها به هم می‌ریزد و IC معمولاً پایین‌تر می‌آید.

پس برنامه چند طول مختلف را امتحان می‌کند و طول‌هایی را جدی‌تر می‌گیرد که گروه‌هایشان IC بالاتری دارند. این روش همیشه پاسخ قطعی نمی‌دهد؛ مثلاً ۶ که مضرب طول واقعی ۳ است نیز ممکن است IC خوبی داشته باشد. به همین دلیل برنامه در کنار IC از آزمون کاسیسکی و امتیازهای دیگر هم استفاده می‌کند.

مثال تقسیم متن به ستون‌ها و محاسبه میانگین IC

فرض کنیم متن انگلیسی زیر با کلید واقعی **DOG** رمز شده است:



THISISASIMPLEENGLISHTEXTWITHREPEATEDENGLISHLETTERFREQUENCIES

متن رمز حاصل، بدون فاصله، چنین است:



```
WVOVWYDGO PDRHSTJZOVVZHLZZWZKFKSSGWSJHBMOWYKZKWHKUTXHEAHBILSY
```

در زمان شکستن رمز، برنامه کلید **DOG** و طول واقعی آن را نمی‌داند. بنابراین طول‌های مختلف را فرض می‌کند و برای هر طول، متن رمز را به ستون‌ها تقسیم می‌کند.

ابتدا فرض کنیم طول کلید **3** است. برنامه حروف را به این شکل پخش می‌کند:



```
حرف ۱ → ستون ۱
حرف ۲ → ستون ۲
حرف ۳ → ستون ۳
حرف ۴ → ستون ۱
حرف ۵ → ستون ۲
حرف ۶ → ستون ۳
و به همین ترتیب ...
```

یعنی ستون اول از حروف جایگاه‌های **1, 4, 7, ...**، ستون دوم از جایگاه‌های **2, 5, 8, ...** و ستون سوم از جایگاه‌های **3, 6, 9, ...** ساخته می‌شود. نتیجه:



```
ستون ۱: WVDPHJVVHZKSWHOKWUHHL
ستون ۲: VWGDSZVLWFSSBWZHTBS
ستون ۳: OYORTOZZZKGJMYKKXAIY
```

حالا IC هر ستون جداگانه محاسبه می‌شود. برای نمونه، در ستون اول ۲۰ حرف داریم و فراوانی حروف تکراری مهم چنین است:



```
دو بار K دو بار و V سه بار W پنج بار H
```

حروفی که فقط یک بار آمده‌اند در صورت کسر IC مقدار $1 \times 0 = 0$ ایجاد می‌کنند. بنابراین صورت کسر ستون اول برابر است با:



```
H: 5×4 = 20
W: 3×2 = 6
V: 2×1 = 2
K: 2×1 = 2

مجموع = 20 + 6 + 2 + 2 = 30
```

مخرج برای ۲۰ حرف:



$$20 \times 19 = 380$$

پس:



$$\text{ستون } ۱ \approx 380 / 30 = \text{IC } 0.0789$$

به همین روش، IC سه ستون به دست می‌آید:

ستون IC

۱ 0.0789

۲ 0.0632

۳ 0.0632

برنامه میانگین آنها را حساب می‌کند:



$$\begin{aligned} &\text{برای طول } ۳ \text{ IC میانگین} \\ &= (0.0789 + 0.0632 + 0.0632) / 3 \\ &\approx 0.0684 \end{aligned}$$

حالا فرض کنیم برنامه طول کلید را 2 حدس بزند. این بار حروف یکی‌درمیان در دو ستون قرار می‌گیرند:



$$\begin{aligned} &\text{ستون } ۱: \text{WOWDODHTZVZLZZFSGSHMWKKHUXEHIS} \\ &\text{ستون } ۲: \text{VVGPRSJOVHZWKKSWJBOYZWKTHABLY} \end{aligned}$$

نتیجه IC:



$$\begin{aligned} &\text{ستون } ۱ \approx \text{IC } 0.0483 \\ &\text{ستون } ۲ \approx \text{IC } 0.0414 \\ &\text{برای طول } ۲ \text{ IC میانگین} \\ &= (0.0483 + 0.0414) / 2 \\ &\approx 0.0448 \end{aligned}$$

خلاصه چند طول فرضی:

طول فرضی کلید تعداد ستون میانگین IC

۰.0448	۲	۲
۰.0684	۳	۳
۰.0476	۴	۴

در میان این سه نامزد، طول 3 میانگین IC بالاتری دارد و مقدار آن به IC معمول زبان انگلیسی، یعنی حدود 0.066، نزدیک است. علت این است که با انتخاب طول درست، هر ستون فقط شامل حروفی است که با یک حرف ثابت کلید رمز شده‌اند. با این حال برنامه فقط به این یک عدد تکیه نمی‌کند و نتیجه را همراه کاسیسکی و امتیازهای دیگر بررسی می‌کند.

۴. مدل آزمون کاسیسکی

اگر یک **n-gram** یکسان در متن اصلی دو بار ظاهر شود و هر دو بار از یک جایگاه دوره‌ای کلید شروع شوند، دنباله کلید استفاده‌شده برای آن‌ها یکسان است؛ پس متن رمز یکسانی تولید می‌کنند. یکسان‌بودن جایگاه دوره‌ای یعنی فاصله دو رخداد بر طول کلید **m** بخش‌پذیر باشد:

$$\text{distance} \equiv 0 \pmod{m}$$

بنابراین عوامل مشترک فاصله سه‌گرام‌ها، چهارگرام‌ها و پنج‌گرام‌های تکراری، نامزدهای خوبی برای طول کلید هستند. این استدلال قطعی نیست، چون بعضی تکرارها می‌توانند تصادفی باشند؛ در نتیجه برنامه امتیاز کاسیسکی را با IC ترکیب می‌کند.

مثال آزمون کاسیسکی با ۳‌گرام، ۴‌گرام و ۵‌گرام

برای دیدن روش، فرض کنیم متن زیر با کلید سه‌حرفی **DOG** رمز شده باشد:

```
متن اصلی:  ATTACK NOWXYZ ATTACK
کلید تکراری: DOGDOG DOGDOG DOGDOG
متن رمز: DHZDQQ QCCAMF DHZDQQ
```

در متن اصلی، کلمه **ATTACK** دو بار تکرار شده است. شروع تکرار دوم ۱۲ حرف بعد از شروع تکرار اول است. چون طول کلید ۳ است و ۱۲ بر ۳ بخش‌پذیر است، هر دو بار **ATTACK** با همان قسمت کلید، یعنی **DOGDOG**، رمز شده‌اند. به همین دلیل هر دو به **DHZDQQ** تبدیل شده‌اند.

متن رمز بدون فاصله چنین است:



DHZDQQCCAMFDHZDQQ

موقعیت حروف را از ۱ شماره گذاری می کنیم:



Position: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18
Letter: D H Z D Q Q Q C C A M F D H Z D Q Q

برنامه ابتدا تمام قطعه های سه حرفی را بررسی می کند. سه گرام های تکراری عبارت اند از:

۲ گرام شروع اول شروع دوم فاصله

۱۲	۱۳	۱	DHZ
۱۲	۱۴	۲	HZD
۱۲	۱۵	۳	ZDQ
۱۲	۱۶	۴	DQQ

برای نمونه، DHZ یک بار از موقعیت ۱ و بار دیگر از موقعیت ۱۳ شروع شده است:



فاصله = 13 - 1 = 12

سپس قطعه های چهار حرفی بررسی می شوند:

۴ گرام شروع اول شروع دوم فاصله

۱۲	۱۳	۱	DHZD
۱۲	۱۴	۲	HZDQ
۱۲	۱۵	۳	ZDQQ

بعد قطعه های پنج حرفی بررسی می شوند:

۵ گرام شروع اول شروع دوم فاصله

۱۲	۱۳	۱	DHZDQ
۱۲	۱۴	۲	HZDQQ

همه قطعه های تکراری فاصله ۱۲ دارند. برنامه عدد هایی را پیدا می کند که ۱۲ بر آنها بخش پذیر باشد:



عوامل های ۱۲: 2, 3, 4, 6, 12

پس این عددها نامزدهای طول کلید هستند. طول واقعی کلید در مثال ما 3 است و در میان این عامل‌ها دیده می‌شود. کاسیسکی به‌تنهایی نمی‌تواند از روی فاصله ۱۲ تشخیص دهد که طول واقعی دقیقاً ۲، ۳، ۴، ۶ یا ۱۲ است. همچنین بعضی قطعه‌ها ممکن است تصادفی تکرار شده باشند. به همین دلیل برنامه تعداد تکرارهای پشتیبان هر طول را می‌شمارد و نتیجه کاسیسکی را با میانگین IC ستون‌ها ترکیب می‌کند.

● ۵. مدل کای‌دو برای بازیابی هر حرف کلید

پس از تعیین طول کلید، هر ستون یک رمز سزار است. برای هر انتقال احتمالی s از ۰ تا ۲۵، ستون رمزگشایی می‌شود. اگر تعداد مشاهده‌شده حرف j برابر O_j و تعداد مورد انتظار آن طبق فراوانی انگلیسی برابر $E_j = N \times p_j$ باشد، امتیاز کای‌دو چنین است:

```
chi_square(s) = sum((O_j - E_j)^2 / E_j)
```

برنامه هر ۲۶ انتقال ممکن را امتحان می‌کند و برای هرکدام یک امتیاز کای‌دو به دست می‌آورد. سپس انتقالی را انتخاب می‌کند که **کمترین امتیاز** را دارد:

```
82.4 = امتیاز انتقال ۰
57.1 = امتیاز انتقال ۱
12.6 = ← کمترین امتیاز امتیاز انتقال ۲
44.8 = امتیاز انتقال ۳
...

best_shift = 2
```

امتیاز کوچک‌تر یعنی فراوانی حروف متن حاصل، شباهت بیشتری به فراوانی معمول زبان انگلیسی دارد. عبارت ریاضی `argmin` فقط شکل کوتاه‌شده «مقداری را انتخاب کن که کمترین امتیاز را تولید می‌کند» بود؛ کد پروژه همین کار را با یک حلقه و مقایسه امتیازها انجام می‌دهد.

مثال عددی کای‌دو

فرض کنیم طول کلید قبلاً مشخص شده و یکی از ستون‌های متن رمز شامل ۹۵ حرف است. متن این ستون با یک انتقال سزار رمز شده است. برای اینکه محاسبه قابل بررسی باشد، متن اصلی و نسخه رمز آن در این مثال چنین‌اند:



Plaintext:

THISISALONGENGLISHTEXTWITHCOMMONLETTERFREQUENCIESANDENOUGHCHARACTERSTOMAKEFREQUENCYANALYSISWORK

Ciphertext column:

WKLVLVDORQJHQJOLVKHAWZLWKFRPPRQOHWWHUIUHTXHQFLHVDQGHQRXJKFKDUDFWHUVWRPDNHIUHTXHQFBDQDOBVLVZRUN

برنامه کلید واقعی را نمی‌داند، بنابراین انتقال‌های ۰ تا ۲۵ را یکی‌یکی امتحان می‌کند. مثلاً با امتحان انتقال ۳، هر حرف سه خانه به عقب برده می‌شود:



W -> T

K -> H

L -> I

V -> S

پس ابتدای متن رمزگشایی شده **THIS...** می‌شود. جدول زیر فقط مربوط به همین حالت آزمایشی، یعنی **Shift = 3** و حرف کلید **D** است. اکنون برنامه فراوانی حروف متن حاصل را با فراوانی استاندارد انگلیسی مقایسه می‌کند. سهم هر حرف در امتیاز کای‌دو با فرمول زیر محاسبه می‌شود:



contribution = ((O - E) ** 2) / E

در این فرمول **O** تعداد مشاهده شده و **E** تعداد مورد انتظار است. چون طول ستون **N = 95** است، برای سه حرف نمونه داریم:

حرف تعداد مشاهده شده **O** فراوانی مرجع **p** تعداد مورد انتظار **E** سهم در کای‌دو

۰٫۰۷۴ ۷٫۷۵۹ ۰٫۰۸۱۶۷ ۷ **A**

۰٫۰۰۰۴ ۱۲٫۰۶۷ ۰٫۱۲۷۰۲ ۱۲ **E**

۰٫۰۴۲ ۸٫۶۰۳ ۰٫۰۹۰۵۶ ۸ **T**

همین محاسبه برای هر ۲۶ حرف انجام و سهم همه حروف با هم جمع می‌شود. امتیاز کامل انتقال ۳ برابر **58.52** است. چند نتیجه دیگر چنین‌اند:

```

Shift 0 (A): 1002.11
Shift 1 (B): 777.09
Shift 2 (C): 439.05
Shift 3 (D): 58.52 <- smallest score
Shift 4 (E): 1219.04
Shift 9 (J): 229.78
Shift 16 (Q): 303.00

```

کمترین امتیاز متعلق به انتقال ۳ است. با قرارداد $A=0$ ، $B=1$ ، $C=2$ و $D=3$ ، برنامه نتیجه می‌گیرد که حرف کلید این ستون D است. همین فرایند برای ستون‌های بعدی تکرار می‌شود و کنار هم قرار گرفتن حروف برنده، کلید کامل ویژنر را می‌سازد.

۶. مدل آنتروپی شانون و فراوانی n-gram

برای توصیف میزان پراکندگی حروف از آنتروپی شانون استفاده می‌شود. اگر احتمال تجربی حرف j برابر $p_j = f_j/N$ باشد:

```

H(X) = -sum(p_j × log2(p_j))

```

اگر فقط یک حرف وجود داشته باشد، احتمال آن ۱ و آنتروپی صفر است. بیشترین آنتروپی برای توزیع یکنواخت ۲۶ حرف رخ می‌دهد و برابر $\log_2(26) \approx 4.70$ بیت بر حرف است. برنامه آنتروپی متن اصلی و متن رمز را محاسبه می‌کند و اختلاف آن‌ها را در یک گزارش JSON می‌نویسد. علاوه بر تک‌حرف‌ها، فراوانی دوگرام‌ها و سه‌گرام‌ها نیز محاسبه می‌شود تا ساختار زبانی متن‌های کاندید بهتر سنجیده شود.

۷. مدل آزمایش نرخ موفقیت

برای هر طول متن L ، حمله T بار روی نمونه‌ها و کلیدهای تصادفی اجرا می‌شود. اگر S بار کلید یا متن دقیقاً بازیابی شود، نرخ موفقیت تجربی برابر است با:

```

success_rate(L) = S / T

```

علاوه بر بررسی بازیابی کامل کلید و متن، برنامه می‌سنجد چند حرف متن در جای درست بازیابی شده‌اند. فرض کنیم طول متن در هر آزمایش $L=10$ و تعداد آزمایش‌ها $T=3$ باشد:



آزمایش اول: 8 حرف درست از 10 حرف $0.8 = 8/10 \rightarrow$
 آزمایش دوم: 10 حرف درست از 10 حرف $1.0 = 10/10 \rightarrow$
 آزمایش سوم: 6 حرف درست از 10 حرف $0.6 = 6/10 \rightarrow$

حالا میانگین سه دقت را می‌گیریم:



```
average_letter_accuracy
= (0.8 + 1.0 + 0.6) / 3
= 0.8
= 80 درصد
```

فرم کلی محاسبه چنین است:



L / دقت هر آزمایش = تعداد حروف درست آن آزمایش
 T / میانگین دقت = مجموع دقت همه آزمایش‌ها

در اینجا L طول متن، T تعداد دفعات آزمایش و `correct_letters_t` تعداد حروف درست در آزمایش شماره t است. این معیار بازیابی ناقص را هم نشان می‌دهد: ممکن است متن کاملاً درست بازیابی نشده باشد، ولی مثلاً ۸۰ درصد حروف آن صحیح باشند.

بنابراین نمودار سه خط `Average letter accuracy`، `Exact plaintext recovery` و `Exact key recovery` دارد. این اعداد برآوردی تجربی از کیفیت حمله برای آن طول متن هستند. استفاده از seed ثابت باعث می‌شود نمونه‌گیری و نمودار آزمایش قابل تکرار باشند. با افزایش طول متن، فراوانی‌های مشاهده‌شده پایدارتر می‌شوند و معمولاً نرخ موفقیت افزایش می‌یابد.

خلاصه روش رمز شکنی

1. فقط حروف A-Z برای محاسبات آماری جدا می‌شوند.
2. میانگین IC ستون‌ها برای طول‌های کاندید محاسبه می‌شود.
3. فاصله n-gram های تکراری در آزمون کاسیسکی فاکتورگیری می‌شود.
4. ترکیب امتیاز IC، کاسیسکی و جریمه پیچیدگی، طول‌های محتمل را رتبه‌بندی می‌کند.
5. هر ستون با آزمون کای دو روی ۲۶ انتقال ممکن شکسته می‌شود.
6. متن‌های کاندید با فراوانی حروف، دوگرام‌ها، سه‌گرام‌ها و کلمات رایج انگلیسی مقایسه می‌شوند.

اجرا

دستورها را داخل پوشه `vigenere_project` اجرا کنید.

برای اجرای ساده و تعاملی، برنامه را بدون آرگومان اجرا کنید:

```
python main.py
```

یک منوی انگلیسی ساده نمایش داده می‌شود و برنامه ورودی‌ها را مرحله به مرحله می‌پرسد. روش‌های خط فرمان زیر نیز برای اجرای مستقیم و قابل تکرار همچنان فعال‌اند.

رمزگذاری:

```
python main.py encrypt --text "Attack at dawn!" --key LEMON
python main.py encrypt --input data/sample_plaintext.txt --key LOGIC --output output/ciphertext.txt
```

رمزگشایی با کلید معلوم:

```
python main.py decrypt --text "Lxfopv ef rnhrl!" --key LEMON
python main.py decrypt --input output/ciphertext.txt --key LOGIC --output output/plaintext.txt
```

شکستن رمز بدون داشتن کلید:

```
python main.py break --input output/ciphertext.txt --max-key-length 15 --output output/recovered.txt
```

ساخت جدول‌های JSON و نمودار حرف/دوگرام/سه‌گرام:

```
python main.py analyze --input output/ciphertext.txt --label ciphertext --output-dir output
```

مقایسه آنتروپی شانون متن اصلی و متن رمز:



```
python main.py compare-entropy --plaintext-input data/sample_plaintext.txt --ciphertext-input  
output/ciphertext.txt --output output/entropy_comparison.json
```

رسم نمودار نرخ موفقیت نسبت به طول متن:



```
python main.py experiment --corpus data/sample_plaintext.txt --lengths 100 200 400 800 1200 --trials 20  
--output-dir output
```

برای نمودارها ابتدا وابستگی را نصب کنید:



```
python -m pip install -r requirements.txt
```

اجرای تست‌ها:



```
python -m unittest discover -s tests -v
```

نکات مهم

- IC متن انگلیسی معمولاً نزدیک ۰/۰۶۶ و متن یکنواخت نزدیک 1/26 است.
- طول‌های مضرب طول واقعی نیز ممکن است IC بالایی داشته باشند؛ به همین دلیل شواهد کاسیسکی، جریمه پیچیدگی و امتیاز زبانی با هم استفاده شده‌اند.
- با کوتاه بودن متن، جدول فراوانی نویزی است و احتمال خطا در طول کلید یا حروف آن بالا می‌رود.
- دو معیار سخت‌گیرانه آزمایش، برابری کامل کلید و متن بازیابی‌شده هستند؛ خط دقت حرفی نیز میزان بازیابی جزئی را جداگانه نشان می‌دهد.
- این پیاده‌سازی کلاسیک روی الفبای انگلیسی A-Z کار می‌کند. فاصله، علائم، عددها و حروف فارسی را تغییر نمی‌دهد و آنها باعث جلو رفتن کلید نمی‌شوند.

پایان گزارش بخش رمز ویژنر